

# Progressive learning: A deep learning framework for continual learning

Haytham M. Fayek<sup>a,\*</sup>, Lawrence Cavedon<sup>b</sup>, Hong Ren Wu<sup>a</sup>

<sup>a</sup> School of Engineering, RMIT University, Melbourne VIC 3001, Australia

<sup>b</sup> School of Science, RMIT University, Melbourne VIC 3001, Australia

## ARTICLE INFO

### Article history:

Received 25 December 2018

Received in revised form 21 March 2020

Accepted 11 May 2020

Available online 18 May 2020

### Keywords:

Continual learning

Computer vision

Deep learning

Machine learning

Neural networks

Speech recognition

## ABSTRACT

Continual learning is the ability of a learning system to solve new tasks by utilizing previously acquired knowledge from learning and performing prior tasks without having significant adverse effects on the acquired prior knowledge. Continual learning is key to advancing machine learning and artificial intelligence. Progressive learning is a deep learning framework for continual learning that comprises three procedures: curriculum, progression, and pruning. The curriculum procedure is used to actively select a task to learn from a set of candidate tasks. The progression procedure is used to grow the capacity of the model by adding new parameters that leverage parameters learned in prior tasks, while learning from data available for the new task at hand, without being susceptible to catastrophic forgetting. The pruning procedure is used to counteract the growth in the number of parameters as further tasks are learned, as well as to mitigate negative forward transfer, in which prior knowledge unrelated to the task at hand may interfere and worsen performance. Progressive learning is evaluated on a number of supervised classification tasks in the image recognition and speech recognition domains to demonstrate its advantages compared with baseline methods. It is shown that, when tasks are related, progressive learning leads to faster learning that converges to better generalization performance using a smaller number of dedicated parameters.

© 2020 Elsevier Ltd. All rights reserved.

## 1. Introduction

Learning from a tabula rasa (blank slate) is the most common machine learning paradigm (Mikolov, Joulin, & Baroni, 2018). For an arbitrary task  $t_k \in \mathcal{T}_\infty = \{t_i \mid i \in \mathbb{Z}^+\}$ , a model  $f_k$  is trained using a dataset  $\mathbb{D}_k$  or environment  $\mathbb{E}_k$  to minimize a loss function  $\ell_k$  or maximize a reward  $r_k$  with the aim of achieving a certain objective  $o_k$ . Contrary to mammalian learning, the model  $f_k$  does not take into account the knowledge learned in prior models  $\mathcal{F}_{k-1} = \{f_i \mid 1 \leq i \leq k-1\}$  for tasks  $\mathcal{T}_{k-1} = \{t_i \mid 1 \leq i \leq k-1\}$  using datasets  $\mathcal{D}_{k-1} = \{\mathbb{D}_i \mid 1 \leq i \leq k-1\}$  or environments  $\mathcal{E}_{k-1} = \{\mathbb{E}_i \mid 1 \leq i \leq k-1\}$ , which may lead to a slower learning process that requires more data and possibly results in suboptimal performance (Lake, Ullman, Tenenbaum, & Gershman, 2017). *Continual learning* is key to advancing machine learning and artificial intelligence, where knowledge and experiences can be accumulated and repurposed over tasks  $\mathcal{T}_\infty$  (Thrun & Mitchell, 1995).<sup>1</sup>

\* Corresponding author.

E-mail address: [haytham.fayek@ieee.org](mailto:haytham.fayek@ieee.org) (H.M. Fayek).

<sup>1</sup> Since the total number of tasks at a given time is known to be finite,  $\mathcal{T}_\infty$  is reduced to  $\mathcal{T}_N = \{t_i \mid 1 \leq i \leq N\}$ .

Continual learning is defined as follows: “The system has performed  $K$  tasks. When faced with the  $(K+1)$ th task, it uses the knowledge gained from the  $K$  tasks to help the  $(K+1)$ th task.” (Thrun, 1996). This poses numerous challenges, two of which are *catastrophic forgetting* and *negative forward transfer*. Catastrophic forgetting (McCloskey & Cohen, 1989; Ratcliff, 1990) refers to the case when the performance of the system for the  $K$  tasks degrades when it learns the  $(K+1)$ th task, as the model forgets the  $K$  tasks to learn the  $(K+1)$ th task. Negative forward transfer refers to the case when the  $K$  tasks have a negative effect on the performance of the system for the  $(K+1)$ th task, which can occur when the  $(K+1)$ th task is not related to any of the  $K$  tasks leading to interfere and worse performance.

A continual learning system requires: (1) a strategy to select the next task to learn from a pool of candidate tasks to facilitate learning; (2) a mechanism to efficiently grow the capacity of the model to accommodate learning new tasks (if necessary); and, (3) a method to accumulate, maintain, and utilize knowledge to learn future tasks without significant adverse effects on the learned tasks.

In this paper, we propose *progressive learning*, a deep learning framework for continual learning that aims to address the aforementioned three desiderata. Progressive learning comprises three procedures: *curriculum*, *progression*, and *pruning*. Curriculum is

used to actively select a subsequent task to learn from a pool of candidate tasks. Progression grows the capacity of the model by adding new parameters, denoted progressive block, that leverage parameters learned in prior tasks to learn from data available for the new task at hand. Only new parameters are trained and existing parameters are not altered, which sidesteps the problem of catastrophic forgetting; however, this leads to growth in the number of parameters as the number of tasks increases. Pruning is used to counteract this growth, where it is used to attempt to remove newly added trained weights in a greedy layer-wise manner without compromising performance. Pruning may also aid in mitigating negative forward transfer as it may allow progressive learning to be reduced to independent learning by removing parameters between progressive blocks, if the new task is not related to prior tasks. Fig. 1 illustrates the progressive learning framework.

Despite relevant work in curriculum (Bengio, Louradour, Collobert, & Weston, 2009; Ruvoilo & Eaton, 2013a), progression (Rusu et al., 2016; Terekhov, Montone, & O'Regan, 2015), and pruning (Han, Pool, Tran, & Dally, 2015; LeCun et al., 1990), there is no deep learning framework for continual learning to point to. The main novel contribution of this paper is the construction and formulation of a deep learning framework that allows a holistic approach to continual learning. Key to the viability of progressive learning is the following:

- The curriculum procedure offers a straight-forward strategy for *active task selection*, i.e., select a task from a pool of candidate tasks, with respect to the current state of knowledge leading to a natural development in task difficulty.
- The progression procedure provides a mechanism to grow the capacity of the model to accommodate learning new tasks, encourages the reuse of features by *concatenating* features learned in prior tasks with features being learned for the task at hand, and avoids catastrophic forgetting.
- The *greedy layer-wise pruning* procedure builds on intuition that the specificity of features varies across layers in deep networks (Fayek, Lech, & Cavedon, 2016; Yosinski, Clune, Bengio, & Lipson, 2014), and therefore aims to adjust the pruning amount per layer to counteract the growth in the number of parameters. Furthermore, the pruning procedure can aid in mitigating negative forward transfer by reducing progressive learning to independent learning via removing parameters between progressive blocks if tasks are unrelated.

Progressive learning is, to the best of our knowledge, the first to package the aforementioned features into a single framework.

Progressive learning is evaluated in two domains: a standard set of image recognition tasks constructed from the CIFAR-10 and CIFAR-100 datasets (Krizhevsky, 2009); and a challenging novel set of speech recognition tasks that include Automatic Speech Recognition (ASR), Speech Emotion Recognition (SER), Speaker recognition (SR), and Gender Recognition (GR) using the TIMIT (Garofolo et al., 1993), IEMOCAP (Busso et al., 2008), and eINTERFACE (Martin, Kotsia, Macq, & Pitas, 2006) datasets. The merits and shortcomings of progressive learning, as well as the contributions of each of its components, are analysed. It is shown that, when tasks are related, progressive learning leads to faster learning that converges to better generalization performance using a smaller number of dedicated parameters compared with independent learning, transfer learning, multi-task learning, and related continual learning baselines.

The remainder of this paper is structured as follows. Section 2 reviews related prior work. Section 3 describes progressive learning. Section 4 presents experiments and results. Section 5 discusses progressive learning and outlines future work. The paper is concluded in Section 6.

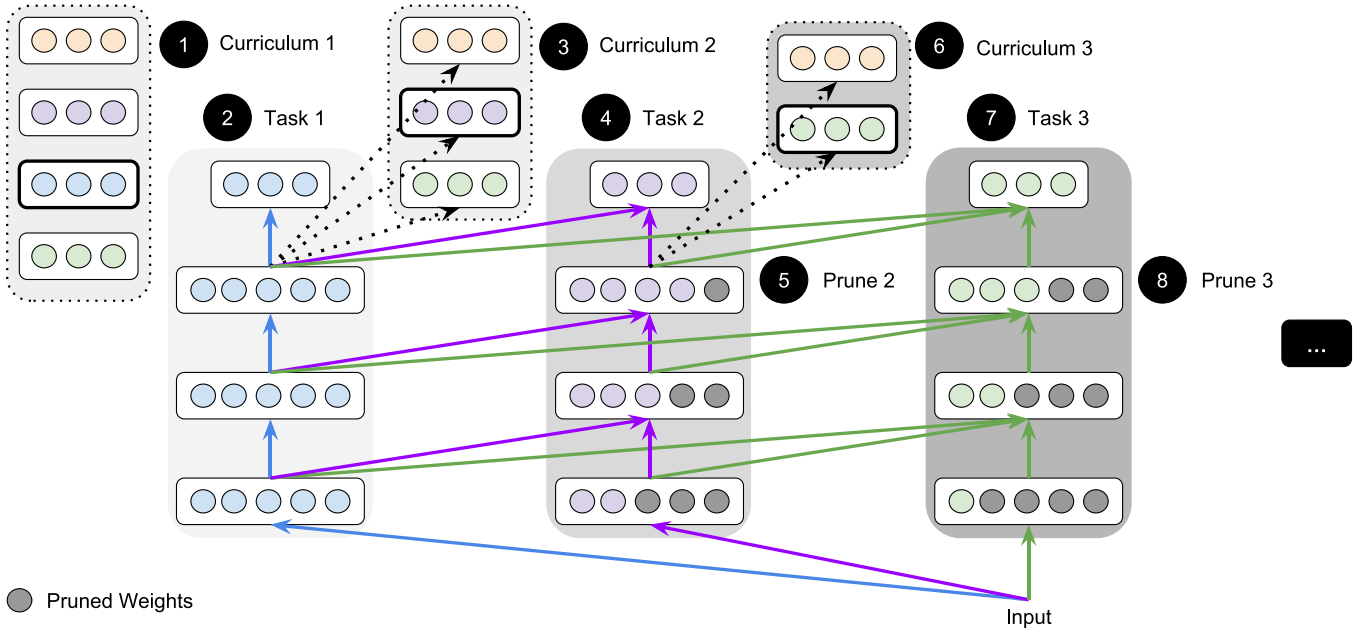
## 2. Related work

Sharing knowledge between tasks has a long history in the field of machine learning. In the context of deep learning and neural networks, paradigms such as *transfer learning* (Pan & Yang, 2010; Taylor & Stone, 2009), *multi-task learning* (Caruana, 1997), and *continual learning* (Chen & Liu, 2016; Mitchell et al., 2018; Ruvoilo & Eaton, 2013b) are alternatives to *independent learning* that can share or repurpose knowledge and experiences over tasks.

*Transfer learning.* Transfer learning incorporates knowledge from a source task into a target task by initializing the parameters of the model for the target task using the parameters learned in the source task, followed by fine-tuning some or all of those parameters (Yosinski et al., 2014). It has been shown to perform better than independent learning in cases where the target task lacks sufficient data. Transfer learning suffers multiple disadvantages from the continual learning perspective. First, prior knowledge is only incorporated at the initialization phase of learning. Second, it is non-trivial to extend the paradigm to utilize multiple source tasks (Misra, Shrivastava, Gupta, & Hebert, 2016). Third, it is susceptible to catastrophic forgetting. Fourth, if both tasks are unrelated to each other, it may lead to negative forward transfer.

*Multi-task learning.* Multi-task learning aims to learn a single model jointly for multiple related tasks by leveraging relevant information contained in the parallel training signals of those tasks (Caruana, 1997; Misra et al., 2016). Multi-task learning also suffers multiple disadvantages from the continual learning perspective. It does not provide a straight-forward mechanism for adding new tasks once the model has been trained, such that it may be required to jointly relearn all tasks whenever a new task is added. Moreover, multi-task learning is intrinsically unable to handle interference between unrelated tasks.

*Continual learning.* Continual learning aims to learn multiple tasks sequentially, e.g., life-long learning (Thrun & Mitchell, 1995) and never-ending learning (Mitchell et al., 2018). As previously mentioned, this leads to the problem of catastrophic forgetting, where the model forgets prior tasks when presented with a new task. Numerous paradigms were developed to overcome catastrophic forgetting as follows. The parameter regularization paradigm aims to find a measure of importance of parameters for prior tasks that can be used to adaptively adjust or penalize their perturbation during training on a new task, and thus mitigate catastrophic forgetting, e.g., elastic weight consolidation (Kirkpatrick et al., 2017) and synaptic intelligence (Zenke, Poole, & Ganguli, 2017). The functional regularization paradigm combats catastrophic forgetting by introducing a regularization term to the loss function that penalizes deviation in the output of the model trained on some prior tasks when being trained on a new task, e.g., learning without forgetting (Li & Hoiem, 2016) and adaptation by distillation (Hou, Pan, Loy, Wang, & Lin, 2018). The architectural paradigm sidesteps catastrophic forgetting by adding new adaptive parameters to the model for each new task, such as block-modular neural networks (Terekhov et al., 2015), progressive neural networks (Rusu et al., 2016), and residual adapters (Rebuffi, Bilen, & Vedaldi, 2017). The experience replay paradigm stores previously seen data from prior tasks, either directly, e.g., gradient episodic memory (Lopez-Paz & Ranzato, 2017), or compressed via a generative model (Shin, Lee, Kim, & Kim, 2017), and utilizes a multi-task objective that combines prior tasks and the new task by replaying previously seen data from prior tasks during training with the introduced new task (Isele & Cosgun, 2018). Finally, the meta-learning paradigm, i.e., learning to learn, can also be used for continual learning, such as in Al-Shedivat et al. (2018).



**Fig. 1.** Overview of progressive learning for three tasks. Initially, a curriculum strategy is used to select a task (blue) from the pool of candidate tasks. Second, a model is trained to perform the selected task (blue), and the learned parameters are constant thereafter. Third, the curriculum strategy is employed to select the subsequent task (purple). Fourth, new model parameters, denoted progressive block, which draw connections from the preceding layer in the block as well as the preceding layer in prior progressive block(s), are added and trained to perform the selected task (purple). Fifth, after training the newly added progressive block to convergence, a pruning procedure is used to remove weights without compromising performance. Finally, the curriculum, progression, and pruning procedures are repeated for the third task (green), and for all remaining task(s) subsequently. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

**Online learning.** Online learning ingests data sequentially, one exemplar at a time; each data exemplar is used to update the model and discarded afterwards, as opposed to batch learning techniques that learn a model using the entire training dataset (Blum, 1998). Online learning is typically employed in the single task setting but it can also be used for multi-task learning (Dekel, Long, & Singer, 2006). Progressive learning is related to growing and pruning algorithms for online learning (Huang, Saratchandran, & Sundararajan, 2005; Savitha, Suresh, & Sundararajan, 2012; Yingwei, Sundararajan, & Saratchandran, 1997), although these algorithms were introduced for single task learning.

**Curriculum.** Curriculum learning follows the notion that a meaningful order of tasks when learning multiple tasks sequentially could significantly facilitate learning (Elman, 1993). Curriculum strategies for training deep neural networks were studied in Bengio et al. (2009) and Graves, Bellemare, Menick, Munos, and Kavukcuoglu (2017), while active task selection was studied in Ruvolo and Eaton (2013a). Therein, models trained with curriculum strategies outperformed the same models trained without curriculum strategies indicating the importance of a structured process to determine a meaningful order of tasks. Note that some studies reported that curriculum strategies that start with easier tasks and progresses towards harder tasks outperformed anti-curriculum strategies that start with harder tasks and progresses towards easier tasks (Bengio et al., 2009; Graves et al., 2017), whereas other studies found the opposite to be true (Ruvolo & Eaton, 2013a).

**Progression.** Block-modular neural networks were proposed in Terekhov et al. (2015), similarly, progressive neural networks were proposed in Rusu et al. (2016). Both approaches were explicitly designed for learning tasks in a sequential manner by training a model for each task that is connected to prior models via adaptive connections. For example, progressive neural networks (Rusu et al., 2016) start with a standard multi-layered neural network trained for some arbitrary task; for every new subsequent task,

an additional multi-layered neural network, denoted block (or column in Rusu et al. (2016)), connected to prior block(s) via lateral connections in addition to its own connections, is added to the network and trained for the new task at hand; all prior block(s) remain constant, thus avoiding catastrophic forgetting. A block is connected to prior block(s) via progressive layers that draw connections from the previous layer in its own block as well as the previous layer in prior block(s), as follows:

$$\hat{\mathbf{y}}_{(k)}^{(l)} = \phi \left( \mathbf{w}_{(k)}^{(l)} \hat{\mathbf{y}}_{(k)}^{(l-1)} + \sum_{j < k} \mathbf{u}_{(k;j)}^{(l)} \hat{\mathbf{y}}_{(j)}^{(l-1)} \right), \quad (1)$$

where  $\hat{\mathbf{y}}_{(k)}^{(l)} \in \mathbb{R}^{n_{lk}}$  is the vector of activations of layer  $l$  in block  $f_k$ ,  $n_{lk}$  is the number of units in layer  $l$  in block  $f_k$ ,  $\phi$  is an element-wise non-linear function,  $\mathbf{W}_{(k)}^{(l)} \in \mathbb{R}^{n_{lk} \times n_{(l-1)k}}$  is the weight matrix of layer  $l$  in block  $f_k$ ,  $\hat{\mathbf{y}}_{(k)}^{(l-1)} \in \mathbb{R}^{n_{(l-1)k}}$  is the vector of activations of the previous layer ( $l-1$ ) in block  $f_k$ ,  $\mathbf{U}_{(k;j)}^{(l)} \in \mathbb{R}^{n_{lk} \times n_{(l-1)j}}$  are the lateral connections from layer ( $l-1$ ) in block  $f_j$  to layer  $l$  in block  $f_k$ , and  $\hat{\mathbf{y}}_{(j)}^{(l-1)} \in \mathbb{R}^{n_{(l-1)j}}$  is the vector of activations of the previous layer ( $l-1$ ) in block  $f_j$ .

**Pruning.** Pruning the parameters of neural networks can not only be used to decrease the computational and memory requirements and speed up training and inference (Han, Mao, & Dally, 2016; Han et al., 2015; Louizos, Welling, & Kingma, 2018), but can also improve generalization (Hassibi & Stork, 1993; LeCun et al., 1990; Louizos et al., 2018; Reed, 1993) as deep neural networks, in particular, are typically over-parametrized, which can easily lead to over-fitting. The pruning procedure used here is an extension to the pruning method proposed in Han et al. (2016, 2015) that relies on training a neural network via standard methods, followed by pruning unimportant parameters, then fine-tuning the remaining parameters of the network to compensate for the pruned parameters.

**Progressive learning.** Progressive learning extends and unifies several key ideas in curriculum, progression, and pruning into a holistic framework for continual learning. The curriculum procedure in progressive learning is a simple method for active task selection that allows for a natural development in task difficulty relative to the state of knowledge, unlike prior work (Ruvolo & Eaton, 2013a), which found anti-curriculum methods to work better. The progression procedure in progressive learning relies on the concatenation operation, which encourages the reuse of features (Huang, Liu, Weinberger, & van der Maaten, 2017), instead of the summation operation used in Rusu et al. (2016). The pruning procedure in progressive learning is carried out in a greedy layer-wise manner, unlike prior work (Han et al., 2015), which prunes the entire network concurrently. These procedures can be regarded as modular building blocks for the progressive learning framework that may be employed individually or in combination as required. Other procedures can be added to progressive learning to address further desiderata of continual learning.

### 3. Progressive learning

Progressive learning is a deep learning framework for continual learning, whereby tasks are learned in sequence with the ability to use prior knowledge from previously learned tasks to facilitate the learning and execution of new ones. The iterative framework comprises three procedures: curriculum, progression, and pruning, which are presented in Sections 3.1, 3.2, and 3.3 respectively.

Intuitively, the curriculum procedure employs a strategy to select the subsequent task to learn from a pool of candidate tasks to facilitate learning, the progression procedure grows the capacity of the model by adding new parameters that leverage parameters learned in prior tasks, while learning from data available for the new task at hand, without being susceptible to catastrophic forgetting, and finally, the pruning procedure is used to counteract the growth in the number of parameters in the progression procedure as further tasks are learned, and can also aid in mitigating negative forward transfer.

Algorithm 1 is a functional decomposition of progressive learning in the supervised learning case, where the objective is to learn a model  $\mathbf{y}_{(k)} = \mathcal{F}_k(\mathbf{x}_{(k)})$  for task  $t_k$  that has a dataset  $\mathbb{D}_k^{(train)}$  composed of  $M_k$  exemplars  $\mathbf{X}_{(k)} \in \mathbb{R}^{M_k \times n_{i_k}}$  and corresponding labels or targets  $\mathbf{Y}_{(k)} \in \mathbb{R}^{M_k \times n_{c_k}}$  such that  $n_{i_k}$  and  $n_{c_k}$  are the dimensionalities of the input and output respectively,  $\mathbf{x}_{(k)} \in \mathbf{X}_{(k)}$ , and  $\mathbf{y}_{(k)} \in \mathbf{Y}_{(k)}$ . Fig. 1 illustrates the progressive learning framework.

#### 3.1. Curriculum

The objective of the curriculum procedure is to determine the most appropriate subsequent task to learn from a pool of candidate tasks with respect to the state of knowledge at the time.

Consider a pool of tasks  $\mathcal{T}_p \subset \mathcal{T}_N$ , where  $\mathcal{T}_N$  is a set of  $N$  classification or regression tasks, a classification or regression model  $g_j$  can be trained and evaluated for each candidate task  $t_j \in \mathcal{T}_p$ , using data available for the respective task  $\mathbb{D}_j$  and features computed from the model learned in prior tasks  $f_{k-1}$ . The performance of each of these classification or regression models  $g_j$  can then be used to measure the relevance of the current features in  $f_{k-1}$  to each of these tasks  $t_j$  (Fayek, Cavedon, & Wu, 2018). The subsequent task  $t_k$  can be chosen using a metric that utilizes the performance of the classification or regression models  $g_j$ . Herein, the task with the highest performance, i.e., the task that can be best solved using the already learned features, is

**Algorithm 1** Functional decomposition of progressive learning in the supervised learning case.

**Require:** Candidate Tasks  $\mathcal{T}_N = \{t_i \mid 1 \leq i \leq N\}$

**Require:** Datasets  $\mathcal{D}_N = \{\mathbb{D}_i = (\mathbf{X}_{(i)}, \mathbf{Y}_{(i)}, t_i) \mid 1 \leq i \leq N\}$

**Output:** Model  $\mathcal{F}_K = \{f_i \mid 1 \leq i \leq K\}$

//  $\mathcal{F}_K$  is a model of  $K$  progressive blocks, each  $f_k$   
// is trained for its respective task  $t_k \in \mathcal{T}_N$  using  $\mathbb{D}_k \in \mathcal{D}_N$

```

1: Initialize:
    $\mathcal{T}_K = \emptyset$  // learned tasks s.t.  $\mathcal{T}_K \subset \mathcal{T}_N$ 
    $\mathcal{D}_K = \emptyset$  // processed datasets s.t.  $\mathbb{D}_K \subset \mathcal{D}_N$ 
    $\mathcal{T}_p \leftarrow \mathcal{T}_N - \mathcal{T}_K$  // remaining tasks s.t.  $\mathcal{T}_p \subset \mathcal{T}_N$ 
    $\mathcal{D}_p \leftarrow \mathcal{D}_N - \mathcal{D}_K$  // remaining datasets s.t.  $\mathbb{D}_p \subset \mathcal{D}_N$ 
    $\mathcal{F}_K = \emptyset$ 

2: for  $k = 1$  to  $N$  do
3:   Curriculum: select next task  $t_k \in \mathcal{T}_p$  // see Section 3.1
4:   Progression: train  $f_k$  on task  $t_k$  using  $\mathbb{D}_k = (\mathbf{X}_{(k)}, \mathbf{Y}_{(k)}, t_k)$  // see Section 3.2
5:   if  $k > 1$  then
6:     Pruning: prune  $f_k$  // see Section 3.3
7:   end if
8:    $\mathcal{T}_p \leftarrow \mathcal{T}_p - t_k$ 
9:    $\mathcal{D}_p \leftarrow \mathcal{D}_p - \mathbb{D}_k$ 
10:   $\mathcal{T}_K \leftarrow \mathcal{T}_K + t_k$ 
11:   $\mathcal{D}_K \leftarrow \mathcal{D}_K + \mathbb{D}_k$ 
12:   $\mathcal{F}_K \leftarrow \mathcal{F}_K + f_k$ 
13: end for
14: return Model  $\mathcal{F}_K$ 

```

chosen as in Eq. (2). Following Eq. (2) is likely to lead to a natural progression in task difficulty relative to the state of knowledge at each iteration.

$$t_k = \underset{t_j \in \mathcal{T}_p}{\operatorname{argmax}} \mathcal{R} \left( g_j \left( f_{k-1} \left( \mathbf{X}_{(j)}; \Theta_{k-1}^{(1:(L_{(k-1)}-1))^*} \right); \psi_j \right); \mathbf{Y}_{(j)} \right) \quad (2)$$

where  $\mathcal{R}$  denotes some measure of positive normalized performance (the larger, the better),  $g_j$  is a classification or regression model parametrized by adaptive parameters  $\psi_j$  trained using  $f_{k-1}(\mathbf{X}_{(j)}; \Theta_{k-1}^{(1:(L_{(k-1)}-1))^*})$  such that parameters  $\Theta_{k-1}^{(1:(L_{(k-1)}-1))^*}$  were trained on some previous task  $t_{k-1}$  and held constant thereafter,  $L_{(k-1)}$  is the total number of layers in block  $f_{k-1}$ ,  $\mathbf{X}_{(j)}$  and  $\mathbf{Y}_{(j)}$  are the training/validation exemplars and labels/targets of task  $t_j$  respectively. When  $k = 1$ ,  $g_j$  can be trained on the data directly.

The curriculum procedure is a dynamic process that evaluates which task to learn at each iteration. For  $N$  tasks, this requires  $N(N+1)/2$  classification or regression models to be trained and evaluated, assuming no new tasks are introduced to the pool of candidate tasks. Training a simple classification or regression model is not as computationally demanding as training a deep model. Further, training and evaluation can be carried out on only a subset of the candidate tasks or using only a subset of the available data for each task.

#### 3.2. Progression

The objective of the progression procedure is to (1) increase the capacity of the model by adding new parameters to accommodate learning a new task; more importantly, (2) provide a mechanism to leverage parameters learned in prior tasks while learning the new task at hand from data available for the task without being susceptible to catastrophic forgetting.

This can be achieved by instantiating and training a new multi-layered neural network for each new task with randomly initialized parameters, denoted *progressive block*, that in addition

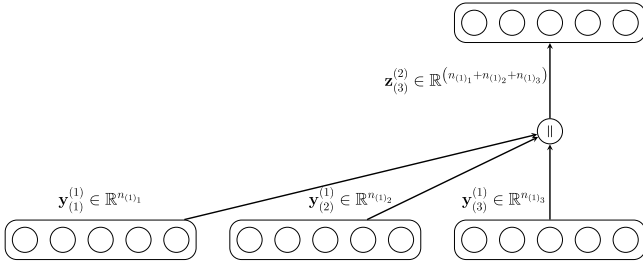


Fig. 2. Illustration of the concatenation operation for three blocks.

to its layer-to-layer connections, draws connections from respective preceding layers in existing progressive blocks, i.e., a layer in a progressive block receives input from the preceding layer in that block, as well as the preceding layer in all prior blocks. The parameters in the newly added progressive block provide the model with additional capacity to learn the new task, while the connections between the prior blocks and the newly added block provide the means to leverage parameters learned in prior tasks.

Progressive learning leads to a model that is formed of multiple progressive blocks that are in turn formed of multiple progressive layers, where initially a standard neural network  $f_1$  is trained on task  $t_1$  using dataset  $\mathbb{D}_1$ , followed by a progressive block  $f_k$  for each subsequent task  $t_k$  that is trained using only dataset  $\mathbb{D}_k$ . Catastrophic forgetting is prevented by only training the newly added parameters of progressive block  $f_k$ , while all parameters in prior blocks remain constant. At inference time for task  $t_k$ , the forward pass of a progressive neural network will propagate through progressive blocks  $\{f_i \mid 1 \leq i \leq k\}$ .

Formally, consider a standard fully connected layer:

$$\hat{\mathbf{y}}^{(l)} = \phi(\mathbf{W}^{(l)} \hat{\mathbf{y}}^{(l-1)}), \quad (3)$$

where  $\hat{\mathbf{y}}^{(l)} \in \mathbb{R}^{n_l}$  is a vector of activations of layer  $l$ ,  $n_l$  is the number of units in layer  $l$ ,  $\phi$  is an element-wise non-linear function such as the Rectified Linear Unit (ReLU),  $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{(l-1)}}$  is a matrix of weights of layer  $l$ ,  $n_{(l-1)}$  is the number of units in the preceding layer  $(l-1)$ ,  $\hat{\mathbf{y}}^{(l-1)} \in \mathbb{R}^{n_{(l-1)}}$  is a vector of activations of the preceding layer  $(l-1)$ ,  $l \in \{1, \dots, L\}$ ,  $L$  is the total number of layers, such that  $\hat{\mathbf{y}}^{(0)}$  is the input to the model, and  $\hat{\mathbf{y}}^{(L)}$  is the output of the model, and the bias term is omitted for clarity. A progressive fully connected layer is defined as follows:

$$\hat{\mathbf{y}}_{(k)}^{(l)} = \phi \left( \mathbf{W}_{(k)}^{(l)} \left( \hat{\mathbf{y}}_{(k)}^{(l-1)} \parallel \left( \bigparallel_{j=1}^{(k-1)} \hat{\mathbf{y}}_{(j)}^{(l-1)} \right) \right) \right), \quad (4)$$

where  $\hat{\mathbf{y}}_{(k)}^{(l)} \in \mathbb{R}^{n_{l,k}}$  is a vector of activations of layer  $l$  in progressive block  $f_k$ ,  $n_{l,k}$  is the number of units in layer  $l$  in progressive block  $f_k$ ,  $\mathbf{W}_{(k)}^{(l)} \in \mathbb{R}^{n_{l,k} \times \sum_{j=1}^k n_{(l-1),j}}$  is a matrix of weights of layer  $l$  in progressive block  $f_k$ ,  $\sum_{j=1}^k n_{(l-1),j}$  is the total number of units in layers  $(l-1)$  in the current and prior progressive blocks,  $\hat{\mathbf{y}}_{(k)}^{(l-1)} \in \mathbb{R}^{n_{(l-1),k}}$  and  $\hat{\mathbf{y}}_{(j)}^{(l-1)} \in \mathbb{R}^{n_{(l-1),j}}$  are vectors of activations of the preceding layer  $(l-1)$  in progressive blocks  $f_k$  and  $f_j$  respectively,  $l \in \{2, \dots, L_k\}$ ,  $L_k$  is the total number of layers in progressive block  $f_k$ , and  $\parallel$  denotes the concatenation operation as illustrated in Fig. 2. When  $k = 1$ , Eq. (4) is reduced to Eq. (3), i.e., the first progressive block  $f_1$  is a standard multi-layered neural network. Eq. (4) can be trivially extended to convolutional and recurrent layers.

Note the important distinction between Eq. (1) (Rusu et al., 2016) and Eq. (4), in that Eq. (4) uses the concatenation operation, whereas Eq. (1) uses a weighted sum of outputs of the previous layers in the previous blocks as well as the current block. The use of the concatenation operation leads to a simpler transformation

that is easier to optimize due to improved gradients flow and encourages the reuse of features (Huang et al., 2017). In particular, the use of the concatenation operation requires only a single weight matrix  $\mathbf{W}_{(k)}^{(l)}$  and a single matrix–vector multiplication (in the case of fully connected layers) for each progressive layer, which is straightforward to parallelize on Graphics Processing Units (GPUs) and convenient to prune as described in Section 3.3. In Section 4.2, particularly Table 2, it is shown that the use of the concatenation operation in Eq. (4), over the summation operation or Eq. (1), leads to improved performance.

### 3.3. Pruning

The objective of the pruning procedure is to counteract the growth in the number of parameters in the model as the number of tasks increases by attempting to remove weights in a progressive block after each progression procedure. Moreover, in most prior work on continual learning, tasks are assumed to be related; if this assumption is invalid, it may lead to negative forward transfer. Herein, the effect of negative forward transfer is mitigated by pruning, which can remove the connections that are responsible for this undesirable transfer, or even reduce progressive learning to independent learning if the new task is not related to any prior task, by removing all connections to prior blocks.

The specificity of features varies across layers in deep networks that tend to learn low-level features in initial layers and transition gradually to high-level more task-specific features towards final layers. Similar low-level features commonly appear across various datasets and tasks, while high-level features are somewhat more attuned to the dataset or task at hand, making initial layers more generic, i.e., easier to use across datasets or tasks. To this end, initial layers in a progressive block are expected to be more prone to pruning than final layers without compromising performance as the model can rely on features learned in initial layers in prior blocks. Therefore, an iterative greedy layer-wise pruning procedure is proposed, which aims to greedily prune the weights in each layer in a progressive block without compromising performance.

The proposed pruning procedure is as follows. Initially, a progressive block  $f_k$  is trained as outlined in Section 3.2 until convergence, using the following loss function:

$$\mathcal{L}_k(\Theta_k) = \ell_k(\Theta_k) + \lambda \frac{1}{2} \sum_{l=1}^{L_k} \|\mathbf{W}_{(k)}^{(l)}\|_2^2, \quad (5)$$

where  $\mathcal{L}_k(\Theta_k)$  is the total loss of progressive block  $f_k$ ,  $\Theta_k$  denotes the model parameters of progressive block  $f_k$  including weights  $\mathbf{W}_{(k)}$ , biases  $\mathbf{b}_{(k)}$ , Batch Normalization (BatchNorm) parameters  $\gamma_{(k)}$ ,  $\beta_{(k)}$ , etc., i.e.,  $\{\mathbf{W}_{(k)}, \mathbf{b}_{(k)}, \gamma_{(k)}, \beta_{(k)}\} \in \Theta_k$ ,  $\ell_k(\Theta_k)$  is the loss function of progressive block  $f_k$  without the weight-decay regularization term  $\frac{1}{2} \sum_{l=1}^{L_k} \|\mathbf{W}_{(k)}^{(l)}\|_2^2$ ,  $\lambda$  is a hyperparameter to trade-off both terms, and  $\|\mathbf{W}_{(k)}^{(l)}\|_2$  is the  $L^2$  norm of the weights of layer  $l$  in progressive block  $f_k$ . The regularization term in the loss function in Eq. (5) is used to encourage the weights of progressive block  $f_k$  to be small in magnitude, and therefore, implicitly encourage the use of previous progressive blocks since their weights do not constitute part of the regularization term. This validates the assumption that weights with small magnitude are not as important as weights with large magnitude, and the magnitude of the weights can be used as a pruning criterion (Han et al., 2016).

Subsequently, after training to convergence, iteratively for each layer, starting from the initial layer to the final layer in progressive block  $f_k$ , the smallest  $q$ -percentile of the sorted magnitudes of the weights of the layer are removed and the entire

**Algorithm 2** Greedy layer-wise pruning procedure in progressive learning.

---

**Require:** Trained progressive block  $f_k$   
**Require:** Pruning step size  $\tilde{q}$   
**Output:** Pruned progressive block  $\tilde{f}_k$

```

1:  $\mathcal{R}_b \leftarrow \text{Evaluate } f_k$  // benchmark performance
2:  $f_b \leftarrow f_k$  // best model
3: for  $l = 1$  to  $L_k$  do // total number of layers  $L_k$  in  $f_k$ 
4:   for  $q = \tilde{q}$  to 100 by  $\tilde{q}$  do
5:     Prune: prune smallest  $q\%$  of weights  $|\mathbf{W}_{(k)}^{(l)}|$  in  $f_k$ 
6:     Train: continue training  $f_k$ 
7:      $\mathcal{R}_c \leftarrow \text{Evaluate } f_k$  // current performance
8:     if  $\mathcal{R}_c \geq \mathcal{R}_b$  then
9:        $\mathcal{R}_b \leftarrow \mathcal{R}_c$ 
10:       $f_b \leftarrow f_k$ 
11:    end if
12:  end for
13:   $f_k \leftarrow f_b$ 
14: end for
15:  $\tilde{f}_k \leftarrow f_b$ 
16: return Pruned progressive block  $\tilde{f}_k$ 

```

---

progressive block continues to train for a small number of iterations, to compensate for the pruned weights, where  $q$  is a hyperparameter. The pruning and training procedures are repeated for an increasingly larger  $q$ , until the entire progressive layer is removed,  $q \rightarrow 100\%$ , and the largest  $q$  that does not lead to degradation in performance is used. The complete pruning procedure is detailed in Algorithm 2.

## 4. Experiments

The evaluation criteria used in this work are outlined in Section 4.1. Progressive learning is analysed on a set of 11 supervised classification tasks in the image recognition domain using the CIFAR-10 and CIFAR-100 datasets in Section 4.2. The tasks in this setting are closely related to each other since 10 out of the 11 tasks are forged from the CIFAR-100 dataset and all 11 tasks have the same image recognition objective, yet the challenge is in the somewhat large number of tasks. Progressive learning is evaluated in the speech recognition domain in Section 4.3, using four supervised classification tasks in the speech recognition domain, namely Automatic Speech Recognition (ASR), Speech Emotion Recognition (SER), Speaker recognition (SR), and Gender Recognition (GR), using the TIMIT (Garofolo et al., 1993), IEMOCAP (Busso et al., 2008), and eNTERFACE (Martin et al., 2006) datasets. This is a challenging setting since the relationship between these tasks is not well understood (Fayek et al., 2016), and moreover the datasets used in these tasks were collected under different conditions.

### 4.1. Evaluation criteria

A number of evaluation criteria were used to assess the performance of the models over the various settings. As standard for the single task setting, the classification accuracy was used to measure the performance of the models in all tasks in the image recognition and speech recognition domains, except for the ASR task, which used the phone accuracy rate (100%–Phone Error Rate (PER)) as the assessment criterion, as commonly employed in ASR. Additionally, the following criteria are used to assess the effectiveness of progressive learning over all tasks involved.

The average accuracy across all tasks as defined in Eq. (6) is used as the primary evaluation criterion.

$$\text{Average Accuracy (AA)} = \frac{1}{K} \sum_{k=1}^K \mathcal{R}_k, \quad (6)$$

where  $K$  is the total number of learned tasks, and  $\mathcal{R}_k$  is the accuracy of task  $t_k$  in progressive learning.

The progressive knowledge transfer, which measures the difference between a task learned with progressive learning and the same task learned independently is defined in Eq. (7) (Lopez-Paz & Ranzato, 2017):

$$\text{Progressive Knowledge Transfer (PKT)} = \frac{1}{K} \sum_{k=2}^K (\mathcal{R}_k - \bar{\mathcal{R}}_k), \quad (7)$$

where  $\bar{\mathcal{R}}_k$  is the accuracy of task  $t_k$  learned independently. Note that progressive knowledge transfer is computed only after the second task  $k = 2$  since the first task  $k = 1$  is learned from a tabula rasa.

The learning speed, that is the performance of the model as a function of training iterations, of tasks learned with progressive learning versus the same tasks learned independently is reported.

The number of free parameters in a progressive block, before and after pruning, and the number of fixed parameters in progressive blocks preceding that block, are also reported to evaluate the effectiveness of the pruning procedure, as well as gauge the number of additional parameters required for additional tasks as the number of tasks increases.

### 4.2. Tasks in the image recognition domain

#### 4.2.1. Datasets and preprocessing

**CIFAR.** The CIFAR-10 and CIFAR-100 datasets (Krizhevsky, 2009) consist of RGB images, of size  $32 \times 32$  pixels, drawn from 10 and 100 classes respectively. Each dataset comprises a training set of 50 000 images and a test set of 10 000 images. Both datasets were used to construct a set of 11 supervised image recognition tasks as follows (Zenke et al., 2017). The CIFAR-10 dataset was used to construct the first classification task, where 45 000 training images drawn from the original CIFAR-10 training set were used as the training set, with the remaining 5000 images held out as a validation set. The 10 000 original CIFAR-10 test set images were used as a test set for this task. The CIFAR-100 was used to construct the remaining 10 classification tasks, each of which was composed of 10 classes drawn from CIFAR-100 100 classes. For each task, the training and validation sets comprised 4500 and 500 images respectively, both of which, were drawn from the original CIFAR-100 training set, and a test set of 1000 images, which were all the test images in the original CIFAR-100 test set that belong to the selected 10 classes of the task. Note that all sets in all tasks are mutually exclusive. The mean and standard deviation of the images were normalized over individual colour channels using the training set statistics for each task individually.

#### 4.2.2. Architectures and training

The architecture of the model used in this image recognition experiment is described in Table 1, ignoring the concatenation operation. The model is a Convolutional Neural Network (ConvNet) that comprises four convolutional layers followed by two fully connected layers, with BatchNorm, ReLUs, and dropout interspersed in-between, and a max pooling layer after the non-linear function of the second and fourth convolutional layers. The final fully connected layer is followed by a softmax function. The parameters of the convolutional and fully connected layers

**Table 1**

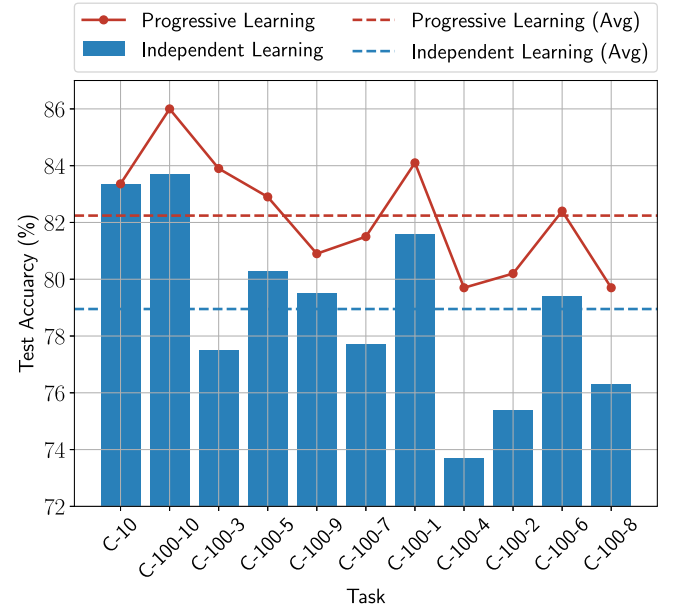
Convolutional neural network architecture for the image recognition tasks. The concatenation operation indicates the layers at which the output of the previous layer in all prior blocks are concatenated. The concatenation operation can be ignored in independent learning.

| No. | Type            | Size             | Other      |
|-----|-----------------|------------------|------------|
| 1   | Convolution     | 32, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                | –          |
|     | ReLU            | –                | –          |
|     | [Concatenation] | –                | –          |
| 2   | Convolution     | 32, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                | –          |
|     | ReLU            | –                | –          |
|     | Max pooling     | $2 \times 2$     | Stride = 2 |
|     | Dropout         | –                | $r = 0.25$ |
|     | [Concatenation] | –                | –          |
| 3   | Convolution     | 64, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                | –          |
|     | ReLU            | –                | –          |
|     | [Concatenation] | –                | –          |
| 4   | Convolution     | 64, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                | –          |
|     | ReLU            | –                | –          |
|     | Max pooling     | $2 \times 2$     | Stride = 2 |
|     | Dropout         | –                | $r = 0.25$ |
|     | [Concatenation] | –                | –          |
| 5   | Fully connected | 512              | –          |
|     | BatchNorm       | –                | –          |
|     | ReLU            | –                | –          |
|     | Dropout         | –                | $r = 0.5$  |
|     | [Concatenation] | –                | –          |
| 6   | Fully connected | 10               | –          |
|     | Softmax         | –                | –          |

were initialized randomly, sampling from a Gaussian distribution, with zero mean and  $\sqrt{2/n_i}$  and  $1/\sqrt{n_i}$  standard deviation for convolutional and fully connected layers respectively, where  $n_i$  is the number of inputs to the layer, as recommended by He, Zhang, Ren, and Sun (2015). All layers were regularized using weight decay with penalty  $\lambda = 0.001$  and dropout with drop probability  $r$  as indicated in Table 1. ADAM was used to optimize the parameters with respect to a negative log-likelihood loss function using batch size  $M_b = 256$ , learning rate  $\alpha = 0.001$ , first moment  $\beta_1 = 0.99$ , and second moment  $\beta_2 = 0.999$  for 90 epochs. These settings are similar to Zenke et al. (2017).

For progressive learning, initially a model that has the same architecture and training recipe outlined above was trained on the first task. An additional progressive block was then added for each subsequent task. Each progressive block had the same architecture with half the number of feature maps/units listed in Table 1 in each layer. The outputs of the layer preceding a convolutional or fully connected layer in all prior blocks were concatenated and fed as input to that convolutional or fully connected layer, as marked [Concatenation] in Table 1. In each progression procedure, the parameters of the new progressive block were randomly initialized, and trained using the same training recipe as described above. Note that parameters in a progressive block are held constant once the block is trained and pruned, including during training and pruning subsequent progressive blocks.

The experiments were implemented using TensorFlow (Abadi et al., 2016). Training was carried out on NVIDIA Tesla P100 GPUs. Training a single model from scratch for a single task on a single GPU required a wall time of approximately 30 mins.



**Fig. 3.** Accuracy of progressive learning vs. independent learning for all 11 image recognition tasks. The tasks are ordered according to the outcome of the curriculum procedure. C-10 denotes the CIFAR-10 task, C-100-s indicates the  $s^{\text{th}}$  CIFAR-100 task.

#### 4.2.3. Results

**Independent baselines.** Baseline models were trained for each task independently, in that 11 models were trained for the 11 image recognition tasks. Table 2 (i–iii) presents the validation and test accuracy of the CIFAR-10 task, the validation and test average accuracy across the 10 CIFAR-100 tasks, and the validation and test average accuracy across all 11 CIFAR-10 and CIFAR-100 tasks. Fig. 3 presents the test accuracy of the models for each of the 11 tasks.

**Multiple tasks baselines.** A single model was then used to learn all 11 tasks sequentially using transfer learning, where a model was trained and evaluated on one task, and the trained parameters were used to initialize the following model that was fine-tuned and evaluated on the subsequent task. This was carried out for all tasks successively, i.e., the model for task  $t_1$  was initialized randomly and the model for task  $t_k$  was initialized using the trained parameters of task  $t_{k-1}$ . Note that the model was evaluated on task  $t_k$  after it was fine-tuned on that particular task only; the performance of the model on task  $t_k$  and all prior tasks would degrade as fine-tuning on the subsequent task  $t_{k+1}$  progressed, a typical example of catastrophic forgetting in transfer learning. The validation and test average accuracy as well as the validation and test progressive knowledge transfer are listed in Table 2 (iv).

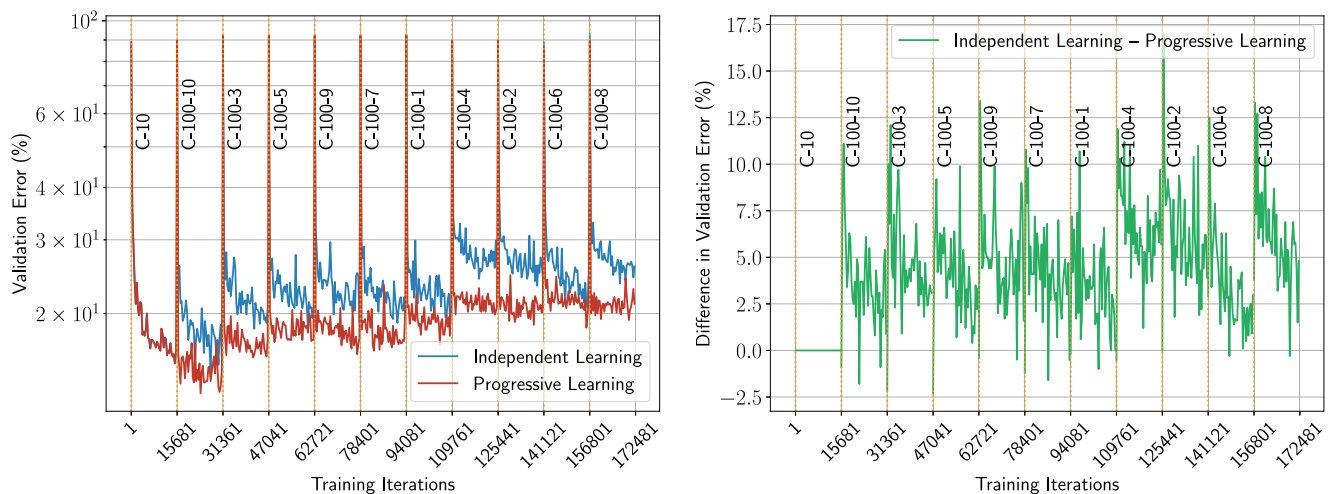
Subsequently, a multi-task learning model was used to learn all 11 tasks in parallel. Two models were used; the first model had a similar architecture to the model listed in Table 1 but with 11 parallel output layers, one for each task; and the second model has a larger architecture with five times the number of parameters as the other model. Due to interference between overlapping classes in the CIFAR-10 and CIFAR-100 datasets, the two previously described models were also trained for the 10 CIFAR-100 tasks only. Table 2 (v–viii) lists the validation and test average accuracy for all multi-task learning models.

Finally, a progressive neural network (Rusu et al., 2016) was used to learn all 11 tasks sequentially. This is conceptually the closest baseline to progressive learning. The validation and test average accuracy as well as the validation and test progressive knowledge transfer are listed in Table 2 (ix). The use of

**Table 2**

Average accuracy over all 11 tasks using the CIFAR-10 and CIFAR-100 datasets unless indicated otherwise. Val AA and Test AA denote the validation and test average accuracy. Val PKT and Test PKT denote the validation and test progressive knowledge transfer.

| #     | Method                                                   | Val AA (%)   | Test AA (%)  | Val PKT (%) | Test PKT (%) |
|-------|----------------------------------------------------------|--------------|--------------|-------------|--------------|
| i.    | CIFAR-10 (Single task)                                   | 84.86        | 83.36        | –           | –            |
| ii.   | CIFAR-100 (Average of 10 Independent Tasks)              | 79.97        | 78.51        | –           | –            |
| iii.  | CIFAR-10 & CIFAR-100 (Average of 11 Independent Tasks)   | 80.41        | 78.95        | –           | –            |
| iv.   | Transfer learning (Average of 11 Independent Tasks)      | 80.23        | 80.39        | –0.2        | 1.58         |
| v.    | Multi-Task Network (Average of 10 CIFAR-100 Tasks)       | 63.32        | 64.24        | –           | –            |
| vi.   | Multi-Task Network (Average of All 11 Tasks)             | 62.69        | 65.80        | –           | –            |
| vii.  | Large Multi-Task Network (Average of 10 CIFAR-100 Tasks) | 67.38        | 66.99        | –           | –            |
| viii. | Large Multi-Task Network (Average of All 11 Tasks)       | 68.38        | 71.26        | –           | –            |
| ix.   | Progressive Neural Networks (Rusu et al., 2016)          | 81.36        | 81.51        | 1.04        | 2.82         |
| x.    | Curriculum + Progression + Pruning                       | <b>83.70</b> | <b>82.24</b> | <b>3.62</b> | <b>3.62</b>  |
| xi.   | Progression                                              | 81.86        | 81.60        | 1.59        | 2.91         |
| xii.  | Progression (Sum)                                        | 78.50        | 78.28        | –2.10       | –0.74        |
| xiii. | Progression + Pruning                                    | 83.96        | 82.18        | 3.91        | 3.56         |
| xiv.  | Progression + Random Pruning                             | 83.77        | 81.99        | 3.70        | 3.34         |
| xv.   | Curriculum + Progression                                 | 81.99        | 81.83        | 1.74        | 3.17         |
| xvi.  | Anti-Curriculum + Progression                            | 81.56        | 81.54        | 1.26        | 2.85         |



**Fig. 4.** Learning curves. Note that tasks are ordered according to the outcome of the curriculum strategy. Models are reset to random initialization at the beginning of each task in the case of independent learning. **Left.** Validation error as a function of training iterations, for progressive and independent learning for the 11 CIFAR-10 and CIFAR-100 tasks. **Right.** Difference in validation error between independent and progressive learning as a function of training iterations for the 11 CIFAR-10 and CIFAR-100 tasks. Progressive learning demonstrates faster learning compared with independent learning.

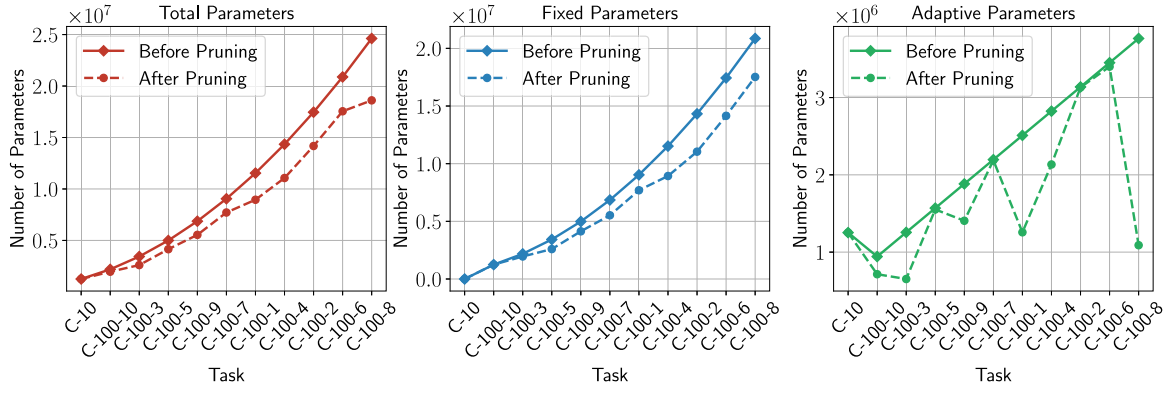
the curriculum, progression, and pruning procedures in progressive learning makes most other continual learning baselines not directly comparable.

Note that all previous models were trained using the same settings described in Section 4.2.2.

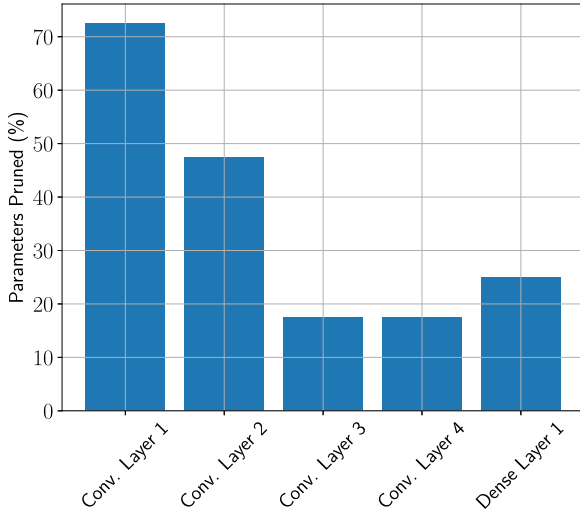
**Progressive learning.** Progressive learning was used to learn the 11 tasks sequentially using the curriculum, progression, and pruning procedures. The order of the tasks was determined via the curriculum procedure that utilized the validation accuracy as its metric. Once a task was chosen, a progression procedure was employed. This was followed by pruning, where the pruning amount  $q$  was varied in 25% intervals, i.e., the pruning amount  $q \in \{25\%, 50\%, 75\%, 100\%\}$ , only for layers that receive input from preceding progressive blocks. The validation set was used to evaluate the performance in the pruning procedure. The validation and test average accuracy as well as the validation and test progressive knowledge transfer are listed in Table 2 (x). The test accuracy for each of the 11 tasks is plotted in Fig. 3. These metrics were measured after learning all 11 tasks. It can be seen from the results in Table 2 and Fig. 3 that progressive learning leads to better performance compared with baseline methods, and in particular independent learning by a large margin.

Progressive learning leverages knowledge learned in prior tasks when learning a new task, and it is therefore expected to learn new tasks faster. Fig. 4 demonstrates the validation error as a function of training iterations, where it is shown that progressive learning converges faster than independent learning.

**Ablation study.** To gain a better understanding of progressive learning, an ablation study was performed, whereby each component of progressive learning was studied separately. Table 2 (xi–xvi) lists the outcome of this ablation study. It can be seen that all three procedures of progressive learning contribute to improvement in performance, with the majority of the contribution obtained from the progression procedure. The following was also observed. First, the presence of a curriculum procedure that chooses the easiest task as the next task leads to an improvement in performance, and outperforms the anti-curriculum strategy that chooses the most difficult task as the next task, as well as the case where no curriculum procedure is employed. Second, the proposed progression procedure based on the concatenation operation outperforms the progression procedure based on the summation operation, and is superior to all baseline models as well as independent learning. Third, the magnitude-based greedy



**Fig. 5.** Total number of parameters in the model as a function of tasks. **Left.** The total number of parameters in the model as a function of tasks before and after pruning. **Centre.** The number of fixed parameters (parameters in preceding progressive blocks) as a function of tasks before and after pruning. **Right.** The number of adaptive parameters (parameters in the progressive block being trained) as a function of tasks before and after pruning.



**Fig. 6.** Percentage of pruned weights as a function of layers. It can be noted that initial layers are more prone to pruning as progressive blocks can rely on features learned in initial layers in preceding progressive blocks.

layer-wise pruning procedure improves performance, in addition to decreasing the number of parameters in the model, and outperforms random greedy layer-wise pruning.

**Analysis.** Progression leads to growth in the number of parameters and pruning was introduced to counteract this growth. As the number of learned tasks increases, the number of pruned parameters is expected to increase, especially in initial layers and with task relatedness. This is shown in Fig. 5 (left), where the difference between the total number of parameters before and after pruning increases as the number of tasks increases. The amount of pruned parameters in each progressive block varied, as shown in Fig. 5 (right), which is likely due to task relatedness or layer-wise feature relevance between tasks.

The amount of pruned parameters is expected to vary between layers. Initial layers typically learn more generic and transferable features, compared with final layers that learn features more attuned to the task at hand. Fig. 6 is a plot of the amount of pruned parameters per layer averaged across all progressive blocks. Fig. 6 demonstrates that indeed the amount of pruned parameters varies between layers; particularly, initial layers are more prone to pruning, since progressive blocks can leverage features learned in initial layers in prior progressive blocks more than features learned in final layers.

### 4.3. Tasks in the speech recognition domain

#### 4.3.1. Datasets and preprocessing

**TIMIT.** The TIMIT dataset (Garofolo et al., 1993) was used for the ASR and GR tasks. The TIMIT dataset contains recordings of 630 speakers, of which 70% are males and 30% are females, from eight major American English dialects, with each speaker reading ten phonetically rich sentences, amounting to a total of 6300 utterances. The complete 462-speaker training set, without the dialect (SA) utterances, was used as the training set. The 50-speaker development set was used as the validation set. The 24-speaker core test set was used as the test set.

In the case of the ASR task, the 61 phonemes were mapped into 48 phonemes for training, which were then mapped into 39 phonemes for scoring, as commonly practised in the literature (Lee & Hon, 1989). The Kaldi toolkit (Povey et al., 2011) was used to produce force-aligned frame labels by training a mono-phone Gaussian Mixture Model (GMM) - Hidden Markov Model (HMM) system with Mel-Frequency Cepstral Coefficients (MFCCs).

In the case of the GR task, a voice activity detector was used to detect and remove silent frames. The ratio between male and female speakers was balanced by randomly omitting utterances from male speakers.

**IEMOCAP.** The IEMOCAP dataset (Busso et al., 2008) was used for the SER task. The dataset comprises 12 h of audio-visual recordings divided into five sessions. Each session is composed of two actors, a male and a female, performing emotional scripts as well as improvised scenarios. Utterances from the first two sessions were used as the training set. The fifth session was used as the validation set. Utterances from the third and fourth sessions were used as the test set. Since the data was labelled at an utterance level, all frames in an utterance inherited the utterance label. A voice activity detector was then used to label silent frames and *silence* was added as an additional class to the four emotion classes, i.e., a frame has either the same label as its parent utterance or the *silence* label. Silent frames were retained in this task as *silence* can be a strong cue for SER.

**eINTERFACE.** The eINTERFACE dataset (Martin et al., 2006) was used for the SR task. The dataset comprises approximately one hour of audio-visual recordings of 44 male and female speakers reading sentences that included a strong emotional component. The training, validation, and test sets were constructed by randomly selecting utterances corresponding to 70%, 15%, and 15% respectively of all data from 40 speakers, while the remaining

**Table 3**

Convolutional neural network architecture for the speech recognition tasks. The concatenation operation indicates the layers at which the output of the previous layer in all prior blocks are concatenated. The concatenation operation can be ignored in independent learning.  $K$  denotes the number of output classes.

| No. | Type            | Size              | Other      |
|-----|-----------------|-------------------|------------|
| 1   | Convolution     | 64, $6 \times 5$  | Stride = 1 |
|     | BatchNorm       | –                 | –          |
|     | ReLU            | –                 | –          |
|     | [Concatenation] | –                 | –          |
| 2   | Convolution     | 64, $3 \times 3$  | Stride = 1 |
|     | BatchNorm       | –                 | –          |
|     | ReLU            | –                 | –          |
|     | Max pooling     | $2 \times 2$      | Stride = 2 |
|     | Dropout         | –                 | $r = 0.25$ |
|     | [Concatenation] | –                 | –          |
| 3   | Convolution     | 128, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                 | –          |
|     | ReLU            | –                 | –          |
|     | [Concatenation] | –                 | –          |
| 4   | Convolution     | 128, $3 \times 3$ | Stride = 1 |
|     | BatchNorm       | –                 | –          |
|     | ReLU            | –                 | –          |
|     | Max pooling     | $2 \times 2$      | Stride = 2 |
|     | Dropout         | –                 | $r = 0.25$ |
|     | [Concatenation] | –                 | –          |
| 5   | Fully connected | 1024              | –          |
|     | BatchNorm       | –                 | –          |
|     | ReLU            | –                 | –          |
|     | Dropout         | –                 | $r = 0.5$  |
|     | [Concatenation] | –                 | –          |
| 6   | Fully connected | $K$               | –          |
|     | Softmax         | –                 | –          |

four speakers were discarded due to inconsistency in their data.<sup>2</sup> Similar to GR, a voice activity detector was used to detect and remove silent frames.

Note that all sets in all tasks are mutually exclusive, except for the ASR and GR tasks, where the training, validation, and test sets overlap between both tasks, i.e., the training set for the ASR is similar to the training set for the GR task, however, the training, validation, and test sets are mutually exclusive in both tasks.

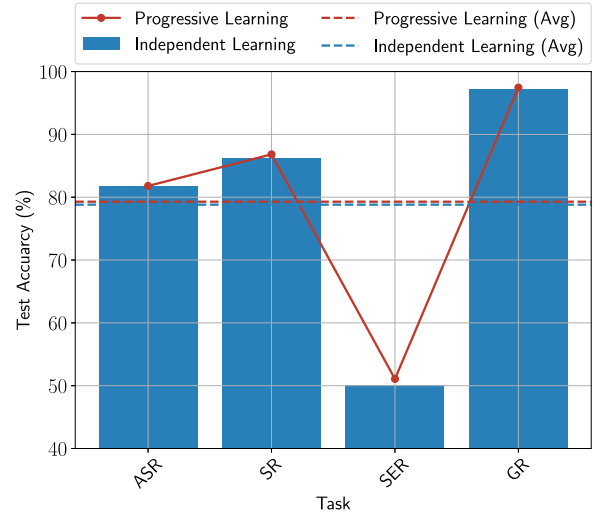
**Preprocessing.** For all datasets and tasks, utterances were split into 25 ms frames with a stride of 10 ms, and a Hamming window was applied, then 40 log Mel Frequency Spectral Coefficients (MFSCs) were extracted from each frame. The mean and standard deviation were normalized per coefficient to zero and one respectively using the mean and standard deviation computed on each training set only. No speaker dependent operations were performed. An input data exemplar was composed of 41 consecutive frames with the exemplar label being the label of the middle frame in the case of ASR and SER or the utterance label in the case of GR and SR.

#### 4.3.2. Architectures and training

The ASR system had a hybrid ConvNet-HMM architecture. A ConvNet acoustic model was used to produce a probability distribution over the states of three-state HMMs with a bi-gram language model estimated from the training set. The other three tasks, namely SER, GR, and SR, had an acoustic model only.

The ConvNet acoustic model used in all four tasks is detailed in Table 3. The model is similar to the model used in Section 4.2 for the image recognition tasks. The parameters of the convolutional and fully connected layers were initialized randomly, sampling

<sup>2</sup> Subjects 6, 11, 12, and 23 had missing or duplicate recordings.



**Fig. 7.** Progressive learning vs independent learning for all four speech recognition tasks. The tasks are ordered according to the outcome of the curriculum procedure.

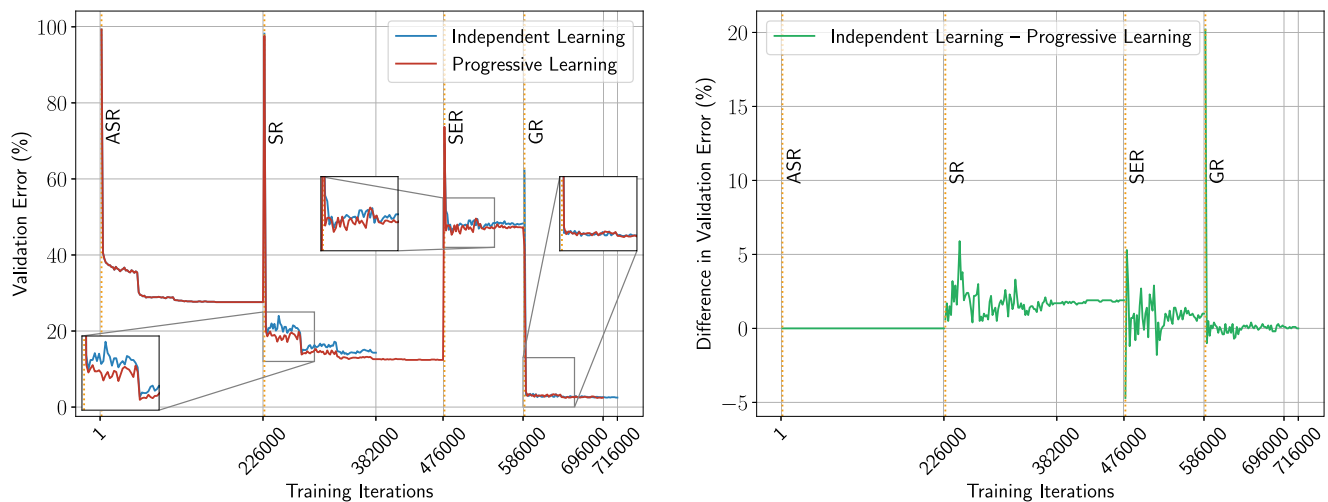
from a Gaussian distribution, with zero mean and  $\sqrt{2/n_i}$  and  $1/\sqrt{n_i}$  standard deviation for convolutional and fully connected layers respectively, where  $n_i$  is the number of inputs to the layer. All layers were regularized using weight decay with penalty  $\lambda = 0.001$  and dropout with drop probability  $r$  as indicated in Table 3. ADAM was used to optimize the parameters with respect to a negative log-likelihood loss function using batch size  $M_b = 256$ , learning rate  $\alpha = 0.001$ , first moment  $\beta_1 = 0.99$ , and second moment  $\beta_2 = 0.999$ . The model was trained for a minimum of  $110 \times 10^3$  iterations and a maximum of  $250 \times 10^3$  iterations. The validation set was used for early stopping.

The experiments were implemented using TensorFlow (Abadi et al., 2016). Training was carried out on NVIDIA Tesla P100 GPUs. Training a single model from scratch for a single task on a single GPU required a wall time of approximately 7 h.

For progressive learning, initially, a model that has the same architecture and training recipe outlined above was trained on the first task. Afterwards, an additional progressive block was added for each subsequent task. Each progressive block had the same architecture with half the number of features maps/units listed in Table 3 in each layer; the outputs of the layer before a convolutional or fully connected layer in all prior blocks were concatenated and fed as input to that convolutional or fully connected layer, as marked [Concatenation] in Table 3. In each progression procedure, the parameters of the new progressive block were randomly initialized, and trained using the same training recipe as mentioned above. Note that parameters in a progressive block are held constant once the block is trained and pruned, including during training and pruning subsequent progressive blocks.

#### 4.3.3. Results

**Baseline.** Baseline models were trained for each task independently, in that four models were trained for the four speech recognition tasks. Fig. 7 plots the test phone accuracy rate for the ASR task, and test accuracy for the SER, GR, and SR tasks. Note that the number of classes should be taken into consideration when assessing the performance of each task: GR is a binary classification task in the TIMIT dataset, SER is a six-class classification task, and SR is a 40-class classification task, whereas the phone accuracy rate is used in ASR.



**Fig. 8.** Learning curves. Note that tasks are ordered according to the outcome of the curriculum strategy. Models are reset to random initialization at the beginning of each task in the case of independent learning. **Left.** Validation error as a function of training iterations, for progressive and independent learning for the four speech recognition tasks. **Right.** Difference in validation error between independent and progressive learning as a function of training iterations for the four speech recognition tasks.

**Progressive learning.** Progressive learning was used to learn the four tasks sequentially, employing the curriculum, progression, and pruning procedures. The curriculum procedure used the relative improvement of the validation accuracy over random guessing to score the classifiers trained at each curriculum procedure. The relative improvement was preferred to the standard validation accuracy as the number of classes varied between tasks. Once a task was chosen, a progression procedure was employed. This was followed by pruning, where the pruning amount  $q$  was varied in 25% intervals, i.e., the pruning amount  $q \in \{25\%, 50\%, 75\%, 100\%\}$  for only layers that receive input from preceding progressive blocks. The validation set was used to monitor and evaluate the performance of the model in the pruning procedure.

The test phone accuracy rate for the ASR task and test accuracy for the remaining three tasks are plotted in Fig. 7. Note that the SER precedes GR in the curriculum procedure due to the use of the relative improvement of the validation accuracy over random guessing to score the classifiers trained at each curriculum procedure as opposed the standard validation accuracy as the number of classes varied between tasks.

It can be seen from the results in Fig. 7 that progressive learning leads to marginally better performance compared with independent learning for all three tasks. To the best of our knowledge, progressive learning is the first to lead to a single model for the speech recognition tasks: ASR, SER, GR, and SR, that outperforms independent models for each task.

Fig. 8 demonstrates the validation error as a function of training iterations, where it is shown that progressive learning converges faster than independent learning on most tasks.

## 5. Discussion and future work

**Continual learning.** Progressive learning is a framework for continual learning that aims to address the following desiderata, as mentioned in Section 1: (1) a strategy to select the next task to learn from a pool of candidate tasks to facilitate learning; (2) a mechanism to efficiently grow the capacity of the model to accommodate learning new tasks (if necessary); and, (3) a method to accumulate, maintain, and utilize knowledge to learn future tasks without significant adverse effects on the learned tasks.

In this work, the aforementioned three desiderata were addressed using three simple yet effective procedures: curriculum,

progression, and pruning. These procedures can be regarded as modular building blocks for the progressive learning framework that may be employed individually or in combination as required. Other procedures can be added to progressive learning to address further desiderata of continual learning.

**Curriculum.** A structured order to learning new concepts and tasks is an important aspect of human learning. It is also beneficial in machine learning (Elman, 1993), and it was shown that it can lead to faster learning, and in some cases, better generalization (Bengio et al., 2009; Graves et al., 2017). The curriculum procedure is an important component of progressive learning, where the learner can choose the most representative task to learn next with respect to the state of knowledge at the time, which leads to a natural development in task difficulty over time. For the tasks studied herein, the curriculum procedure led to better generalization as shown in Table 2.

**Progression.** The progression procedure is the main procedure in progressive learning that has the largest contribution in terms of performance as shown in Table 2. The progression procedure increases the capacity of the model by adding new parameters, and provides a mechanism for forward knowledge transfer, i.e., use knowledge learned in prior tasks to facilitate the learning and execution of new tasks, by leveraging existing parameters learned in prior tasks, all while sidestepping catastrophic forgetting, as only newly added parameters are trained. Existing model parameters are leveraged by concatenating the output of the preceding layer in all prior progressive blocks as well as the current block and feeding the concatenated output to the subsequent layer in the block. The use of the concatenation operation, as opposed to other operations, encourages the reuse of features, and also leads to easier optimization, as pointed out in Huang et al. (2017). Growing the capacity of the model by adding new parameters is important in the case when the new task is not related to any of the prior tasks, which would allow progressive learning to be reduced to independent learning via pruning parameters between progressive blocks.

**Pruning.** The pruning procedure in progressive learning offers many advantages. It was initially conceived to counteract the growth in the number of parameters as a result of the progression procedure. Nevertheless, it was also found to combat over-fitting and negative forward transfer, which can lead to an improvement in performance as shown in Table 2. To this end, pruning

was favoured to incremental growing, where the parameters would grow incrementally in the progression procedure. Layer-wise greedy pruning was built on the intuition that features in initial layers in deep networks are more generic and reusable across tasks. It is envisioned that as the number of tasks increases and with a diverse enough set of tasks, pruning would be able to eliminate most initial layers in new progressive blocks.

**Limitations.** The current progression procedure provides a mechanism for forward knowledge transfer. One of the limitations of this work is the absence of a mechanism to allow backward knowledge transfer, i.e., transfer knowledge obtained from a new task being learned to prior tasks already learned. It is envisioned that this could be achieved by modifying the progression procedure to cater for backward knowledge transfer in addition to forward knowledge transfer by incorporating methods introduced in Lopez-Paz and Ranzato (2017) and Kirkpatrick et al. (2017). This is an area for future research.

## 6. Conclusion

Continual learning is key to advancing machine learning, where information and experiences can be accumulated and used across tasks. This paper formulated progressive learning, a deep learning framework for continual learning that comprises three procedures: curriculum, progression, and pruning.

Progressive learning was evaluated using supervised classification tasks in the image recognition and speech recognition domains, where it was shown that progressive learning outperforms independent learning, transfer learning, multi-task learning, and related continual learning baselines, in terms of overall performance as well as learning speed when tasks are related, by accumulating and leveraging knowledge learned across tasks in a continual manner.

It is envisioned that progressive learning is a step towards a fully general continual learning framework.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

H. M. Fayek is funded by the Vice-Chancellor's Ph.D. Scholarship (VCPS) from RMIT University. This research was undertaken with the assistance of resources and services from the National Computational Infrastructure (NCI), which is supported by the Australian Government. The authors gratefully acknowledge the support of NVIDIA Corporation with the donation of one of the Tesla K40 GPUs used for this research.

## References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). Tensorflow: A system for large-scale machine learning. In *USENIX symposium on operating systems design and implementation*, Vol. 16 (pp. 265–283).
- Al-Shedivat, M., Bansal, T., Burda, Y., Sutskever, I., Mordatch, I., & Abbeel, P. (2018). Continuous adaptation via meta-learning in nonstationary and competitive environments. In *International conference on learning representations*.
- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. In *International conference on machine learning* (pp. 41–48). New York, NY, USA: ACM.
- Blum, A. (1998). On-line algorithms in machine learning. In A. Fiat, & G. J. Woeginger (Eds.), *Online algorithms: The state of the art* (pp. 306–325). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Busso, C., Bulut, M., Lee, C.-C., Kazemzadeh, A., Mower, E., Kim, S., et al. (2008). IEMOCAP: interactive emotional dyadic motion capture database. *Language Resources and Evaluation*, 42(4), 335–359.
- Caruana, R. (1997). Multitask learning. *Machine Learning*, 28(1), 41–75.
- Chen, Z., & Liu, B. (2016). *Lifelong machine learning*. Morgan & Claypool Publishers.
- Dekel, O., Long, P. M., & Singer, Y. (2006). Online multitask learning. In G. Lugosi, & H. U. Simon (Eds.), *Learning theory* (pp. 453–467). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Elman, J. L. (1993). Learning and development in neural networks: the importance of starting small. *Cognition*, 48(1), 71–99.
- Fayek, H. M., Cavedon, L., & Wu, H. R. (2018). On the transferability of representations in neural networks between datasets and tasks. In *Continual learning workshop, 32nd neural information processing systems*, Montréal, Canada.
- Fayek, H. M., Lech, M., & Cavedon, L. (2016). On the correlation and transferability of features between automatic speech recognition and speech emotion recognition. In *Interspeech*, San Francisco, CA, USA (pp. 3618–3622).
- Garofolo, J. S., Lamel, L. F., Fisher, W. M., Fiscus, J. G., Pallett, D. S., Dahlgren, N., et al. (1993). TIMIT acoustic-phonetic continuous speech corpus. *Linguistic Data Consortium*, 93.
- Graves, A., Bellemare, M. G., Menick, J., Munos, R., & Kavukcuoglu, K. (2017). Automated curriculum learning for neural networks. In *International conference on machine learning*, Vol. 70, Sydney, NSW, Australia (pp. 1311–1320).
- Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International conference on learning representations*.
- Han, S., Pool, J., Tran, J., & Dally, W. J. (2015). Learning both weights and connections for efficient neural networks. In *Advances in neural information processing systems* (pp. 1135–1143). Cambridge, MA, USA: MIT Press.
- Hassibi, B., & Stork, D. G. (1993). Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in neural information processing systems* (pp. 164–171).
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *International conference on computer vision* (pp. 1026–1034). Washington, DC, USA: IEEE Computer Society.
- Hou, S., Pan, X., Loy, C. C., Wang, Z., & Lin, D. (2018). Lifelong learning via progressive distillation and retrospection. In *European conference on computer vision* (pp. 452–467). Springer.
- Huang, G., Liu, Z., Weinberger, K. Q., & van der Maaten, L. (2017). Densely connected convolutional networks. In *IEEE conference on computer vision and pattern recognition*.
- Huang, G.-B., Saratchandran, P., & Sundararajan, N. (2005). A generalized growing and pruning RBF (GGAP-RBF) neural network for function approximation. *IEEE Transactions on Neural Networks*, 16(1), 57–67.
- Isele, D., & Cosgun, A. (2018). Selective experience replay for lifelong learning. In *AAAI conference on artificial intelligence*.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
- Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images: Technical Report*, University of Toronto.
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2017). Building machines that learn and think like people. *Behavioral and Brain Sciences*, 40.
- LeCun, Y., Boser, B. E., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W. E., et al. (1990). Handwritten digit recognition with a back-propagation network. In *Advances in neural information processing systems* (pp. 396–404).
- Lee, K.-F., & Hon, H.-W. (1989). Speaker-independent phone recognition using hidden Markov models. *IEEE Transactions on Acoustics, Speech and Signal Processing*, 37(11), 1641–1648.
- Li, Z., & Hoiem, D. (2016). Learning without forgetting. In *European conference on computer vision* (pp. 614–629). Springer.
- Lopez-Paz, D., & Ranzato, M. A. (2017). Gradient episodic memory for continual learning. In *Advances in neural information processing systems* (pp. 6467–6476). Curran Associates, Inc..
- Louizos, C., Welling, M., & Kingma, D. P. (2018). Learning sparse neural networks through  $L_0$  regularization. In *International conference on learning representations*.
- Martin, O., Kotsia, I., Macq, B., & Pitas, I. (2006). The eNTERFACE'05 audio-visual emotion database. In *International conference on data engineering workshops*. Washington, DC, USA: IEEE Computer Society.
- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24, 109–165.
- Mikolov, T., Joulin, A., & Baroni, M. (2018). A roadmap towards machine intelligence. In *Computational linguistics and intelligent text processing* (pp. 29–61). Springer.
- Misra, I., Shrivastava, A., Gupta, A., & Hebert, M. (2016). Cross-stitch networks for multi-task learning. In *IEEE conference on computer vision and pattern recognition* (pp. 3994–4003).
- Mitchell, T., Cohen, W., Hruschka, E., Talukdar, P., Yang, B., Betteridge, J., et al. (2018). Never-ending learning. *Communications of the ACM*, 61(5), 103–115.
- Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359.

- Povey, D., Ghoshal, A., Boulianne, G., Burget, L., Glembek, O., Goel, N., et al. (2011). The kaldi speech recognition toolkit. In *Workshop on automatic speech recognition and understanding*. IEEE Signal Processing Society.
- Ratcliff, R. (1990). Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2), 285–308.
- Rebuffi, S.-A., Bilen, H., & Vedaldi, A. (2017). Learning multiple visual domains with residual adapters. In *Advances in neural information processing systems* (pp. 506–516).
- Reed, R. (1993). Pruning algorithms—a survey. *IEEE Transactions on Neural Networks*, 4(5), 740–747.
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., et al. (2016). Progressive neural networks. CoRR, [abs/1606.04671](https://arxiv.org/abs/1606.04671).
- Ruvolo, P., & Eaton, E. (2013a). Active task selection for lifelong machine learning. In *AAAI conference on artificial intelligence* (pp. 862–868). AAAI Press.
- Ruvolo, P., & Eaton, E. (2013b). ELLA: An efficient lifelong learning algorithm. In *International conference on machine learning* (pp. 507–515).
- Savitha, R., Suresh, S., & Sundararajan, N. (2012). A meta-cognitive learning algorithm for a fully complex-valued relaxation network. *Neural Networks*, 32, 209–218.
- Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual learning with deep generative replay. In *Advances in neural information processing systems* (pp. 2990–2999).
- Taylor, M. E., & Stone, P. (2009). Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research (JMLR)*, 10(Jul), 1633–1685.
- Terekhov, A. V., Montone, G., & O'Regan, J. K. (2015). Knowledge transfer in deep block-modular neural networks (pp. 268–279). Springer.
- Thrun, S. (1996). Is learning the *n*th thing any easier than learning the first? In *Advances in neural information processing systems* (pp. 640–646). Cambridge, MA, USA: MIT Press.
- Thrun, S., & Mitchell, T. M. (1995). Lifelong robot learning. *Robotics and Autonomous Systems*, 15(1), 25–46.
- Yingwei, L., Sundararajan, N., & Saratchandran, P. (1997). A sequential learning scheme for function approximation using minimal radial basis function neural networks. *Neural Computation*, 9(2), 461–478.
- Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?. In *Advances in neural information processing systems* (pp. 3320–3328). Curran Associates, Inc..
- Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. In *International conference on machine learning*, Vol. 70, Sydney, NSW, Australia (pp. 3987–3995).